

What's up with utf8?

David M. Jones

June 25, 2026

We'll cover this in more detail next week, but in the meantime, I hope this will clarify some of the points I tried to make yesterday.

Roughly speaking, an *encoding* is a map from some finite subset of integers $[0..n] = \{0, 1, \dots, n\}$ to a set of characters. The actual number of characters encoded could be less than $n + 1$ since there could be holes in the encoding.

Some examples:

- ASCII is an encoding that assigns 128 characters to the integers $[0..127]$.
- ISO-8859-1 is an encoding that assigns 256 characters to the integers $[0..255]$.
- Unicode is an encoding that (as of version 17.0.0) assigns 154,998 characters to integers in the range $[0..1,114,111]$.

These encodings have the property that each is a strict extension of the previous one. That is, all three encodings agree on which characters they assign to each integer in $[0..127]$, and Unicode and ISO-8859-1 agree on which characters they assign to integers in $[128, 255]$.

Note that I have deliberately not said anything about bits or bytes yet because at this level of abstraction, all that matters is integers.

Of course, as soon as we want to store characters in a computer, we have to grapple with the fact that computers don't deal in integers; they deal in bits. And more than that, they like it when bits come in sets of 8 (i.e., in bytes). And more than *that*, if they need to group bytes together, they prefer to do it in groups of powers of 2. That is, computers are happy dealing with groups of 2 bytes or 4 bytes or 8 bytes, etc., but not with groups of 3 bytes or 6 bytes.

So, the first question you have to ask about an encoding is "How many bits do I need to be able to represent an arbitrary character in this encoding?" A moment's thought says the answer is

$$\text{num bits} = \lfloor \lg(n + 1) \rfloor + 1$$

where $\lg$ is the base 2 logarithm. A quick calculation shows that for our three sample encodings, the number of bits we need is

	bits
ASCII	7
ISO-8859-1	8
Unicode	21

But since computers deal with bytes in groups of powers of 2, we see that the minimum number of bytes needed to represent an arbitrary character in each encoding is

	bytes
ASCII	1
ISO-8859-1	1
Unicode	4

This means that the natural way to represent either ASCII or ISO-8859-1 text is with a *fixed-width* representation where each character is represented by the same number of bytes, namely 1.

Note the following obvious point: If a file only contains bytes whose numeric values are in the range [0..127], that file is both a valid ASCII file and a valid ISO-8859-1 file *and* it represents the same series of characters in both encodings.

We could represent Unicode with a fixed-width representation where each character takes up 4 bytes, but this would be wasteful. For one thing, since we only need 21 bits to represent the maximum Unicode code point, at least 11 bits of each quartet would be 0.

The situation is even worse for documents that consist entirely of ASCII or ISO-8859-1 characters, since in that case fully 3/4 of each quartet would consist of 0 bits.

So, instead, the most popular way to represent Unicode is UTF-8, which is a *variable-width* encoding where each Unicode character is represented by between 1 and 4 bytes.¹

UTF-8 has the property that any character in the ASCII range is represented by a single byte, and that byte has the same value as the ASCII code point. In other words, if a file consists entirely of bytes whose values are in the range [0..127], it is simultaneously a valid ASCII file, a valid ISO-8859-1 file, and a valid UTF-8 file, and it represents the same series of characters in all three encodings.²

But what if it contains bytes in the range [128..255]? Well, it's clearly not ASCII, but it could be either ISO-8859-1 or UTF-8. There are some restrictions on which bytes can occur in UTF-8 and in which bytes can occur next to each other,³ so sometimes you can rule out UTF-8 based on that.

However, any sequences of bytes that can occur in a valid UTF-8 file can also occur in a valid ISO-8859-1 file (since *any* sequence of bytes can occur in an ISO-8859-1 file), so you can never rule out the possibility that a file is ISO-8859-1 based solely on which bytes it contains.

You may have noticed that I've been throwing around the word "valid" rather liberally. Unfortunately, there's no reason why a file can't contain an inconsistent mixture of encodings, such as a solitary 8-bit byte inserted in the middle of a file that otherwise uses UTF-8. Often this results in the entire file being interpreted as ISO-8859-1. You've all seen this: it's what results in, for example, an á appearing as Ã¡.⁴ This can also happen if you feed a UTF-8 file to a program such as pdf_{tex} that doesn't understand UTF-8 and treats the file as series of bytes.

Another thing to be aware of is that if you're working in an environment that has adequate support for Unicode, you shouldn't have to worry about encoding issues except when reading or writing a file. Once you've read some data, ideally you should be able to ignore the encoding. For example, in Perl if you open a file for reading using the UTF-8 encoding layer, Perl translates the string into an internal format that allows you to match on, for example `\x{00E1}` instead of `\x{00C3}\x{00A1}`. And if you

¹See the Wikipedia page 'UTF-8' for the details

²Naturally, it's also a valid ISO-8859-2 file and a valid ISO-8859-3 file, etc., and represents the same sequence of characters in each of those encodings. But it's also a valid EBCDIC file and represents a different sequence of characters.

³For example, a byte in the range [128, 255] can't stand alone; it must be either preceded by or followed by another byte in that same range. Incidentally, this is one of the things I got wrong yesterday; I said there were some code points in the range [128..255] that were represented by a single byte in UTF-8, which is not true.

⁴Something like this might be behind the behaviour that Charles was describing with respect to prd-iso2tex, but I need to see some examples to be sure.

then open a file for output with, say, the UTF-16BE encoding and write that same data to it, Perl will automatically re-encode it in UTF-16BE.

Something similar happens with GNU Emacs, but with extra magic. If you don't explicitly specify the encoding to use when reading the file, Emacs will apply some heuristics to decide what encoding to use. The heuristics are good, but not perfect. And of course, there's not much they can do if the file uses an inconsistent combination of encodings.